



南開大學
Nankai University

人工智能技术实验 实验报告

实验名称: A* 算法的实现

姓 名: 殷家兴

学 号: 2313666

专 业: 自动化

人工智能学院

2025 年 11 月

一 背景问题——寻路问题

寻路问题 (Pathfinding Problem) 是人工智能和计算机科学中的经典问题, 广泛应用于游戏 AI、机器人导航、自动驾驶等领域。该问题的核心是在包含障碍物的环境中, 从起点到终点找到一条代价最小的可行路径。

一般通过将地图切割成网格、节点和边的方式把寻路问题转化为图上的最短路径搜索问题。在此基础上, 可以采用不同的搜索策略进行求解, 包括无信息搜索 (如广度优先搜索 BFS、深度优先搜索 DFS) 和有信息搜索 (如启发式搜索)。

然而, 无信息搜索方法虽然能保证找到最优解, 但其搜索范围大、效率低, 尤其在大规模地图中容易导致计算资源消耗过高。为此, 引入启发式信息的搜索算法成为更优选择。

二 问题简述

A* 算法一种典型的启发式搜索算法, 是静态路网中求解最短路径最有效的方法之一。启发式搜索又称为有信息搜索, 它是利用问题拥有的启发信息来引导搜索, 达到减少搜索范围、降低问题复杂度的目的, 通常拥有更好的性能。A* 算法关键部分为启发函数:

$$F = G + H \quad \text{or} \quad f'(n) = g'(n) + h'(n) \quad (1)$$

其中, G 为从起点 A 移动到当前方格的代价, H 是该方格到终点 B 的估算成本, F 为寻找下一个遍历节点的依据。

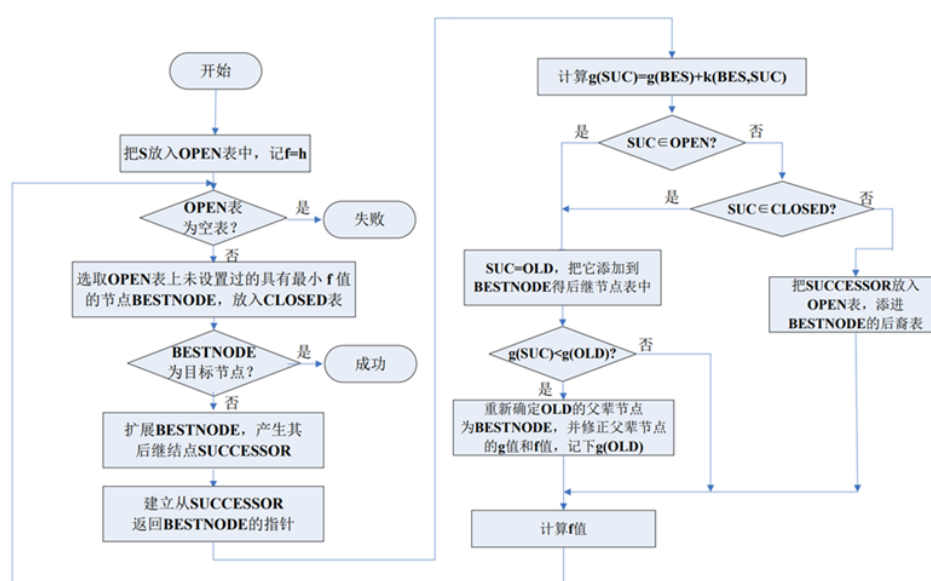


图 1: 代码流程图

伪代码如下：

```
1 初始化 open_list 和 close_list;
2 * 将起点加入 open_list 中，并设其移动代价为 0;
3 * 如果 open_list 不为空，则从 open_list 中选取移动代价最小的节点 n:
4   * 如果节点 n 为终点，则：
5     * 从终点开始逐步追踪 parent 节点，一直达到起点；
6     * 返回找到的结果路径，算法结束；
7   * 如果节点 n 不是终点，则：
8     * 将节点 n 从 open_list 中移除，并加入 close_list 中；
9     * 遍历节点 n 所有的邻近节点：
10      * 如果邻近节点 m 在 close_list 中，则：
11        * 跳过，选取下一个邻近节点
12      * 如果邻近节点 m 在 open_list 中，则：
13        * 计算起点经 n 到 m 的 g 值
14        * 如果此时计算出来的 g 值小于原来起点经 m 的原父节点到 m
          的 g 值：
15          * 更新 m 的父节点为 n，重新计算 m 的移动代价  $f=g+h$ 
16          * 否则，跳过
17      * 如果邻近节点 m 不在 open_list 也不在 close_list 中，则：
18        * 设置节点 m 的 parent 为节点 n
19        * 计算节点 m 的移动代价  $f=g+h$ 
20        * 将节点 m 加入 open_list 中
```

三 实验目的

1. 掌握图搜索算法思路和流程，理解无信息搜索与有信息搜索的区别；
2. 对于启发式搜索，理解 A* 算法估值函数的选取对算法性能的影响；
3. 对于启发式搜索，理解 A* 算法求解流程和搜索顺序；
4. 深入理解对图形化界面设计。（选做）

四 实验内容

1. 实现 Dijkstra 算法 ($H=0$)，以带有障碍物的二维地图为基础，寻找到达目标点的最短路径；
2. 实现 A* 算法，以带有障碍物的二维地图为基础，寻找到达目标点的最短路径；

3. 对 Dijkstra 算法和 A* 算法的性能进行比较分析;
4. 对实验进行图形化界面设计, 展示搜索过程和最优路径。(选做)

五 实验步骤

5.1 Dijkstra 算法实现

Dijkstra 算法是一种无信息搜索算法, 通过逐步确定从源点到图中各个顶点的最短路径来工作。在每一步中, 它都会选择一个当前距离源点最近的、且尚未处理过的顶点, 然后通过这个顶点来更新其所有邻居顶点到源点的距离估计。与 A* 算法相比, 当 $h'(n) = 0$ 时, A* 算法退化为 Dijkstra 算法, 虽然仍然能解得最优解但是运行效率最低。

编写的 Dijkstra 算法 Python 代码如下:

```
1 def dijkstra_search(grid, start, goal):
2     # 获取地图网格行列数
3     rows, cols = len(grid), len(grid[0])
4     # 存储待考察的节点
5     open_list = []
6     # g'(n) 节点n距离起点的代价
7     g_score = {start: 0}
8     came_from = {}
9     # 存储已考察完毕的节点
10    close_list = set()
11    heapq.heappush(open_list, (0, start[0], start[1]))
12    # 待考察网格相对坐标和H, (x, y, G); 上下左右移动代价为10, 对
    角线移动代价为14
13    directions = [
14        (1, 0, 10), (-1, 0, 10), (0, 1, 10), (0, -1, 10),
15        (1, 1, 14), (1, -1, 14), (-1, 1, 14), (-1, -1, 14)
16    ]
17    while open_list:
18        current_g, r, c = heapq.heappop(open_list)
19        current = (r, c)
20        if current in close_list:
21            continue
22        close_list.add(current)
23
24        # 如果当前网格是目标网格, 则回溯路径
25        if current == goal:
```

```

26     path = []
27     while current in came_from:
28         path.append(current)
29         current = came_from[current]
30     path.append(start)
31     path.reverse()
32     return g_score[goal], path
33
34     # 探索相邻网格
35     for dr, dc, cost in directions:
36         n_r, n_c = r + dr, c + dc
37         if 0 <= n_r < rows and 0 <= n_c < cols and grid[n_r][n_c]
           == 0:
38             neighbor = (n_r, n_c)
39             # 计算相邻网格的g'(n)值
40             tentative_g = g_score[current] + cost
41             # 如果当前网格的g'(n)值小于其他相邻网格的g'(n)值, 则
               更新最优路径
42             if neighbor not in g_score or tentative_g < g_score[
               neighbor]:
43                 g_score[neighbor] = tentative_g
44                 came_from[neighbor] = current
45                 heapq.heappush(open_list, (tentative_g, n_r, n_c
               ))
46
47     return float('inf'), [] # 无路径

```

5.1.1 A* 算法实现

在 Dijkstra 算法代码实现的基础上增加 $h'(n)$ 完成对 A* 算法的实现。

```

1 def manhattan(p1, p2):
2     """曼哈顿距离"""
3     return abs(p1[0] - p2[0]) + abs(p1[1] - p2[1])
4
5 def euclidean(p1, p2):
6     """欧氏距离"""
7     return math.sqrt((p1[0] - p2[0])**2 + (p1[1] - p2[1])**2)
8
9 def chebyshev(p1, p2):
10    """切比雪夫距离"""
11    return max(abs(p1[0] - p2[0]), abs(p1[1] - p2[1]))

```

```

10
11 def A_star_search(grid, start, goal, heuristic_type='manhattan'):
12     rows, cols = len(grid), len(grid[0])
13
14     if heuristic_type == 'manhattan':
15         h_func = lambda a, b: manhattan(a, b) * 10
16     elif heuristic_type == 'euclidean':
17         h_func = lambda a, b: euclidean(a, b) * 10
18     elif heuristic_type == 'chebyshev':
19         h_func = lambda a, b: chebyshev(a, b) * 10
20     else:
21         raise ValueError("heuristic_type must be 'manhattan', 'euclidean', or 'chebyshev'")
22
23     open_list = []
24     g_score = {start: 0}
25     # 增加节点n距离终点的预计代价h'(n), f'(n)=g'(n)+h'(n)
26     f_score = {start: h_func(start, goal)}
27     came_from = {}
28     close_list = set()
29     heapq.heappush(open_list, (f_score[start], 0, start[0], start
30                               [1]))
31     directions = [
32         (1, 0, 10), (-1, 0, 10), (0, 1, 10), (0, -1, 10),
33         (1, 1, 14), (1, -1, 14), (-1, 1, 14), (-1, -1, 14)
34     ]
35     while open_list:
36         _, current_g, r, c = heapq.heappop(open_list)
37         current = (r, c)
38         if current in close_list:
39             continue
40         close_list.add(current)
41         if current == goal:
42             path = []
43             while current in came_from:
44                 path.append(current)
45                 current = came_from[current]
46             path.append(start)
47             path.reverse()

```

```

47         return g_score[goal], path, g_score, f_score
48
49     for dr, dc, cost in directions:
50         n_r = r + dr
51         n_c = c + dc
52         if 0 <= n_r < rows and 0 <= n_c < cols and grid[n_r][n_c]
           ] == 0:
53             neighbor = (n_r, n_c)
54             tentative_g = g_score[current] + cost
55             if neighbor in close_list:
56                 continue
57             # 更新路径为当前这个更近更优的方案
58             if neighbor not in g_score or tentative_g < g_score[
               neighbor]:
59                 came_from[neighbor] = current
60                 g_score[neighbor] = tentative_g
61                 h = h_func(neighbor, goal)
62
63                 #  $f'(n) = g'(n) + h'(n)$ 
64                 f = tentative_g + h
65
66                 f_score[neighbor] = f
67                 heapq.heappush(open_list, (f, tentative_g, n_r,
                    n_c))
68     return float('inf'), [], {}, {}

```

这里对 $h'(n)$ （节点 n 距离终点的预计代价）的计算采取三种不同的计算方法，具体如下：

- 曼哈顿距离：从当前格到目的格之间水平和垂直的方格的数量总和。

$$h(s) = |s.x - goal.x| + |s.y - goal.y| \quad (2)$$

- 欧氏距离：两个点之间的真实距离（任意方向移动）。

$$c = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2} \quad (3)$$

- 切比雪夫距离：各坐标数值差绝对值的最大值。

$$h(s) = \max(|s.x - goal.x|, |s.y - goal.y|) \quad (4)$$

5.2 A* 算法可视化

对于 A* 算法，通过可视化窗口的方式实现起终点变化、障碍物选取与 $h'(n)$ 求解方法切换等功能，并将路径规划结果直观地显示出来。

可视化部分的具体代码略过，这里展示可视化窗口界面及对应功能。

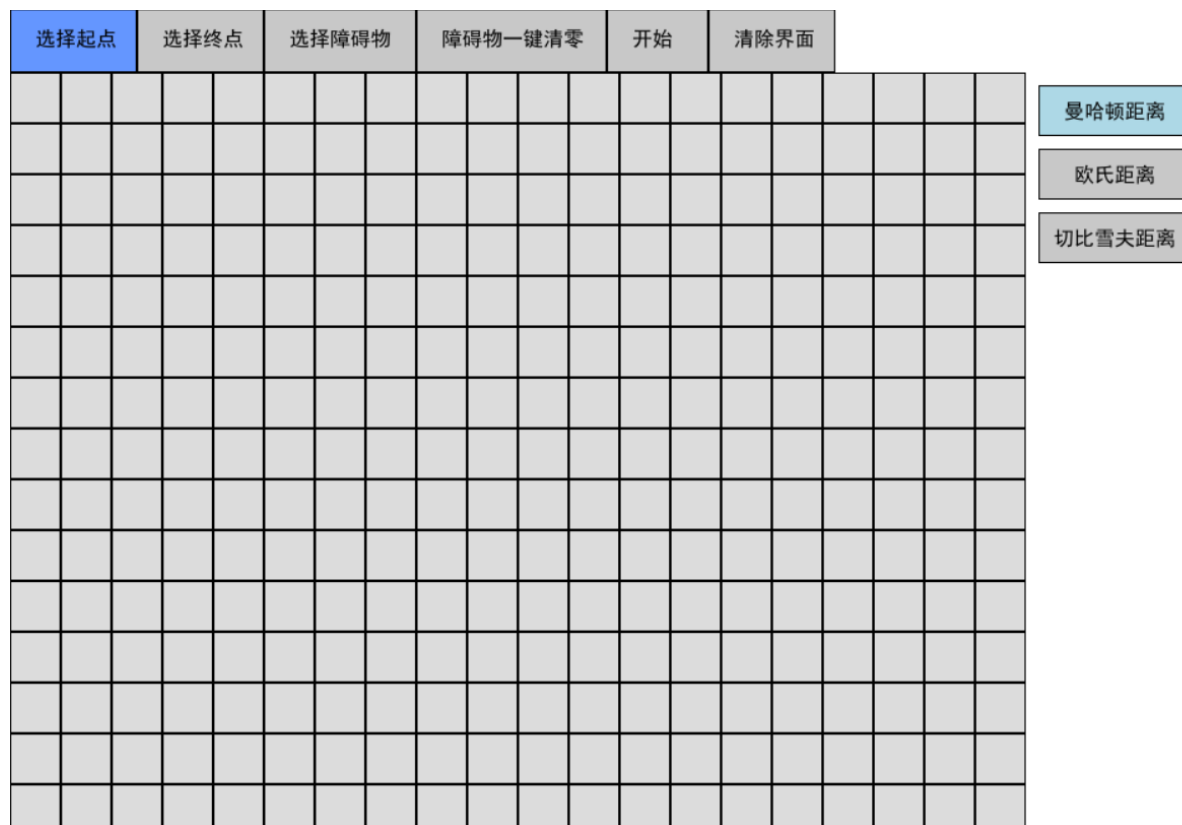


图 2: 可视化窗口

- **$h'(n)$ 计算方法选取：**代码默认选择曼哈顿距离作为 $h'(n)$ 的计算方法，可以通过鼠标点击右侧三种计算方法进行切换。
- **选择起终点：**初始默认鼠标点击动作为选择起点，可以在地图网格中任意点击进行选择，起点选择后鼠标点击动作自动切换为终点选择。如果需要更换起终点可以通过点击上方“选择起点”和“选择终点”窗口重新选择。
- **障碍物选取：**完成终点选择后鼠标点击动作自动切换为障碍物选择，可以多次点击地图网格构建起不同的环境地图。如果需要重新绘制障碍物，可以通过点击“障碍物一键清零”按钮清除目前所有障碍物。
- **路径规划：**完成障碍物绘制后可以通过点击“开始”按钮或直接按下“Enter/回车”键开始路径规划，等待路径结果显示。如果无最终结果，则出现提示窗口。



图 3: 提示窗口

- **界面清除：**完成上一次路径规划后可以通过点击“清除界面”按钮实现界面一键清空，进而开始下一次起终点选取、障碍物绘制与路径规划。

六 实验结果

6.1 Dijkstra 算法运行结果

通过二维列表绘制地图及障碍物：

```

1  # 地图
2  grid = [
3      [0, 0, 0, 1, 0],
4      [0, 1, 0, 1, 0],
5      [0, 1, 0, 0, 0],
6      [0, 0, 0, 1, 0],
7      [0, 0, 0, 0, 0]
8  ]
9  # 起点
10 start = (0, 0)
11 # 终点
12 goal = (4, 4)

```

最终输出计算结果为：

Dijkstra 结果:

总代价: 62

路径: [(1, 1), (1, 2), (2, 3), (3, 4), (4, 5), (5, 5)]

6.2 A* 算法运行结果

与 Dijkstra 算法相同, 通过二维列表实现对地图的绘制。这里选择与 Dijkstra 算法运行结果部分中相同的地图与起终点。

输出计算结果为:

A* 结果:

总代价: 62

路径: [(1, 1), (1, 2), (2, 3), (3, 4), (4, 5), (5, 5)]

节点 $f'(n) = g'(n) + h'(n)$ 验证:

(1,1): $g'(n)=0$, $h'(n)=80$, $f'(n)=80$

(1,2): $g'(n)=10$, $h'(n)=70$, $f'(n)=80$

(2,3): $g'(n)=24$, $h'(n)=50$, $f'(n)=74$

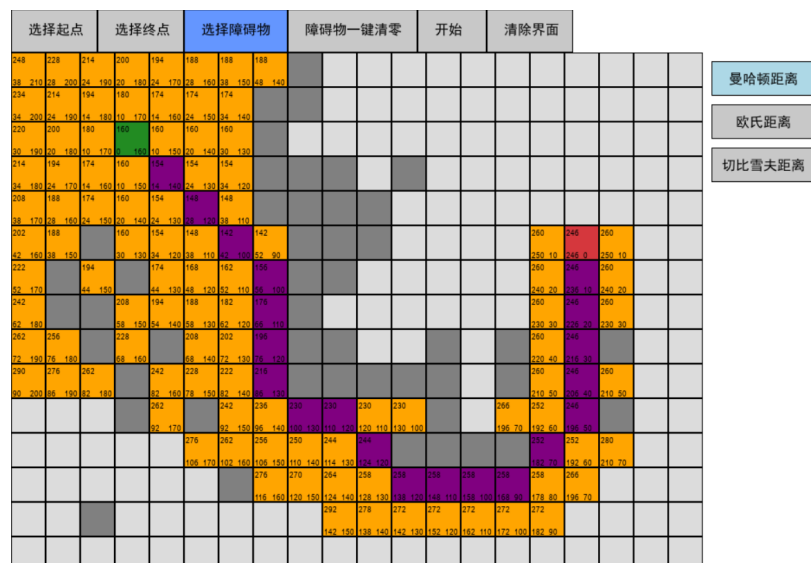
(3,4): $g'(n)=38$, $h'(n)=30$, $f'(n)=68$

(4,5): $g'(n)=52$, $h'(n)=10$, $f'(n)=62$

(5,5): $g'(n)=62$, $h'(n)=0$, $f'(n)=62$

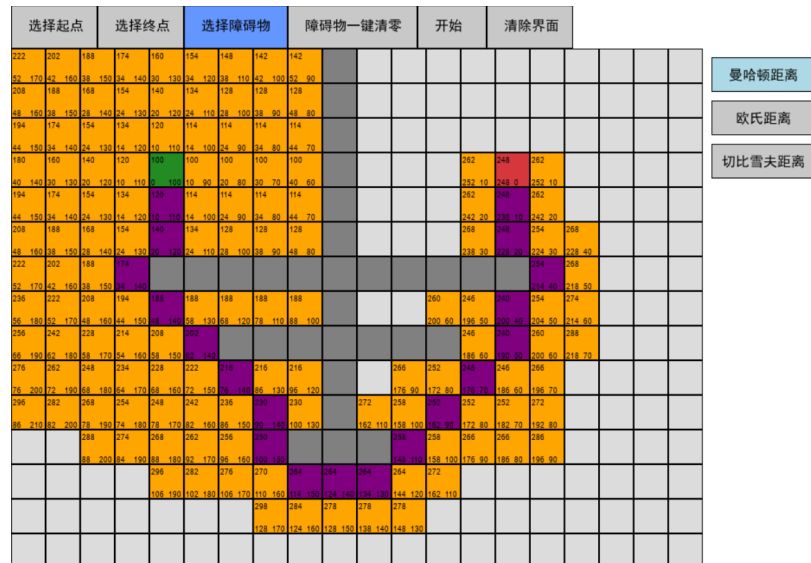
6.3 A* 算法可视化运行结果

任意选取起终点与障碍物, 选择曼哈顿距离作为 $h'(n)$ 计算方法, 结果如下:



切换不同 $h'(n)$ 计算方法，对比各方法求解效果：

- 曼哈顿距离：



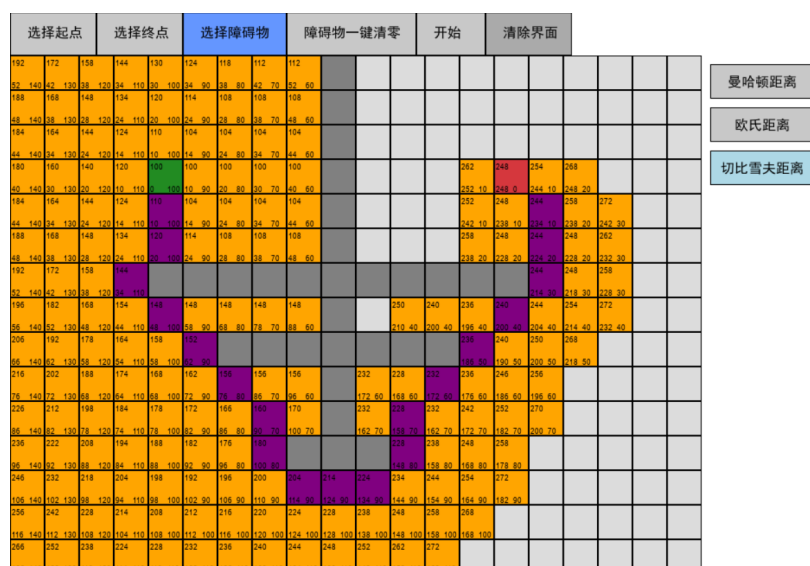


图 7: 使用切比雪夫距离计算结果

可以看到，三种计算方法最终得到的 $f'(n)$ 大小相同。但是，采用曼哈顿距离求解路径时更偏向于规划直角路线；采用欧氏距离和切比雪夫距离求解路径时相对更加自然。进一步测试并查阅相关资料，得到对比结果如表 1 所示。

表 1: 曼哈顿距离、欧氏距离与切比雪夫距离方法对比

$h'(n)$ 计算方法	搜索效率	路径质量
曼哈顿距离	中等	偏向于直角路径
欧氏距离	最低	最自然
切比雪夫距离	最高	偏向于对角线

七 分析总结

本次实验完成了对 Dijkstra 算法和 A* 算法的实现，并探究了不同启发函数 ($h'(n)$) 对 A* 算法性能的影响：曼哈顿距离计算简单，但路径规划偏向于直角；欧氏距离估计最准确，路径自然但计算效率低；切比雪夫距离搜索效率最高。

在此次实验中，我加深了对 Dijkstra 算法和 A* 算法的理解。并且，对不同启发函数的对比分析，为我在优化机器人自主导航与路径规划模块的工作中提供了新的思路。